Thetawave.ai

💻 指令模式 – 設計與應用實戰

Generated by Thetawave.ai

🕹️ 指令模式(Command Pattern)介紹與應用場景

> **指令模式**的核心是將「每一道指令封裝成一個物件」，使得指令的發出者（Invoker）與接收者（Receiver）能夠解耦，並提升程式的彈性與可維護性。

實際生活比喻

指令模式類似將遙控器上的每個按鈕視為「指令插槽」，而每個指令是一個獨立的「指令物件」。按下按鈕時，遙控器只負責呼叫對應的指令物件執行，不需要知道指令的具體執行細節。

🏠 遙控器案例說明

初版需求與問題點

按鈕編號	操作說明	控制家具
0	電風扇調整到下一個風力等級	電風扇(Fan)
1	電風扇調整到上一個風力等級	電風扇(Fan)
2	冷氣開啟	冷氣(Air Conditioner)
3	冷氣關閉	冷氣(Air Conditioner)
4	電視開啟	電視(Television)
5	電視關閉	電視(Television)

- 遙控器(Controller)中使用 switch case 直接寫死按鈕與操作的對應關係。

- **問題**：當需求改變或新增家具時，程式碼會爆炸（修改難度大，不易維護）。

ThinkWave.ai

🔄 第二版需求：彈性綁定指令

- 希望**能夠在外部輕易更改按鈕與指令的綁定規則**，符合**開閉原則 (Open-Close Principle, OCP)**。
- 不允許修改遙控器內部的程式碼，只能透過新增或替換指令物件來實現新功能。

🧩 指令模式的解決方案

核心思想

> **將每一道具體指令封裝成一個物件（Concrete Command），遙控器只負責呼叫這個指令物件的執行方法（Execute），不直接操作家具（Receiver）。**

對應角色

角色名稱	說明	遙控器案例中實體
Client	建立並設定指令物件	Main Method 中實例化指令與家具
Invoker	發出指令（呼叫指令物件）	Controller 類別（遙控器）
Command	指令介面，定義執行方法	Command Interface，含 Execute 方法
Concrete Command	具體指令類別，實作 Command 介面	Turn On AC Command, Fan Next Level Command 等
Receiver	真正執行動作的對象	Furniture 物件，如 Air Conditioner, Fan, Television

---

Thetawave.ai

🛠️ 指令模式實作三步驟

步驟	說明
1. 封裝變動處	將 6 道不同家具指令從 Controller 中抽離，封裝成獨立類別
2. 抽象共同行為	萃取出 Command 介面，定義統一的 Execute() 方法供所有指令實作
3. 委派責任	Controller 不再直接控制家具，改為持有 Command 陣列，按鈕執行對應 Command 的 Execute()

📌 指令介面與具體指令範例

```
// 指令介面
interface Command {
  void Execute();
}

// 具體指令範例：電風扇調整到下一個風力等級
class FanNextLevelCommand implements Command {
  private Fan fan;
  public FanNextLevelCommand(Fan fan) { this.fan = fan; }
  public void Execute() { fan.nextLevel(); }
}
```

- 每個具體指令都**依賴注入(Dependency Injection)**其控制的家具物件。
- Execute()方法封裝了具體的操作細節。

🔘 Controller 類別改寫

功能	描述

指令陣列	持有 Command 陣列，對應 6 個按鈕的指令插槽
設定指令方法	提供 SetCommand(int buttonIndex, Command command) 讓 Client 設定按鈕指令
按鈕執行方法	PressButton(int buttonIndex) 呼叫該按鈕對應的 Command 的 Execute()

- 不再需要知道家具細節，與具體操作完全解耦。
- 方便替換、擴充指令物件。

💡 指令模式的設計優點

優點	說明
解耦 Invoker 與 Receiver	遙控器不認識家具物件，只知道 Command 介面
符合開閉原則 (OCP)	新增或修改指令不需更改遙控器程式碼
易於維護	各指令封裝獨立，錯誤可快速定位於具體指令類別
支援還原(Undo)與重做(Redo)	在 Command 介面新增 Undo() 方法，實作指令的反向操作，搭配堆疊結構實現 Undo/Redo 功能

↩️ ↪️ Undo 與 Redo 功能設計

Undo/Redo 演算法核心

規則編號	規則說明
1	執行新指令後，將該指令推入堆疊 S1，並清空堆疊 S2
2	Undo 時，從 S1 彈出指令，呼叫該指令的 Undo()，再推入堆疊 S2
3	Redo 時，從 S2 彈出指令，呼叫該指令的 Execute()，再推入堆疊 S1

Undo 舉例

Thetawave.ai

- **Turn On Air Conditioner Command** 的 Execute() 是「開啟冷氣」
- 對應的 Undo() 是「關閉冷氣」

🏷️ 指令模式關鍵詞總整理

名稱	中文解釋	角色功能與說明
Command	指令介面	定義執行指令的介面，通常含有 Execute() 方法
Concrete Command	具體指令類別	實作 Command，封裝具體的動作與 Receiver
Invoker	調用者（如遙控器）	呼叫 Command 的 Execute()，不關心具體動作
Receiver	接收者（被控制家具）	實際執行指令動作的物件，如冷氣、電風扇、電視
Client	用戶端	建立指令、接收者與調用者，並設定指令與按鈕的綁定
依賴注入(DI)	依賴注入	將 Receiver 物件注入 Concrete Command，促進鬆耦合
開閉原則(OCP)	對修改封閉、對擴充開放	新增指令不需修改既有程式碼，提高系統彈性

🎯 指令模式的應用時機與意義

- **當應用程式中有「操作」與「指令」需要彈性綁定時，指令模式是一個理想解決方案。**
- 透過將指令封裝成物件，能有效降低系統耦合度，提升擴充性與維護性。
- 系統可以輕易實現複雜功能，如還原（Undo）、重做（Redo）、排程指令等。

Thetawave.ai

指令模式透過將指令封裝為物件，讓調用者與執行者徹底解耦，促使系統具備良好彈性與維護性。這種結構使得系統能隨時新增或替換指令，且能自然實現還原與重做功能，符合軟體設計中追求的模組化與可擴充性。